

Finanzas App

node **>=18.0.0** pnpm **8.15.0** license **MIT** TypeScript **5.9**

Aplicación web de gestión financiera personal diseñada para procesar y analizar extractos bancarios de forma eficiente. Construida con React, Express y Prisma en arquitectura monorepo.

Descripción

Finanzas App es una herramienta que permite cargar extractos bancarios (Excel/CSV), categorizar transacciones de forma semi-automática, y generar reportes analíticos por períodos (trimestres/años). Diseñada para un uso personal o pequeños equipos que necesitan llevar control detallado de sus finanzas.

Principales características:

- Carga de archivos Excel/CSV con preview y revisión antes de guardar
- Dashboard analítico con filtros por período, categoría e ítem
- Asignación manual y masiva de ítems a transacciones
- Gráficos de distribución por categoría y análisis temporal
- Generación de reportes PDF personalizados
- Búsqueda y filtrado avanzado de transacciones históricas

Tech Stack

Frontend

- Framework: [React 19](https://react.dev/) + [TypeScript](https://www.typescriptlang.org/)
- Build Tool: [Vite 7](https://vite.dev/)
- Routing: [React Router 7](https://reactrouter.com/)
- State Management:
 - [TanStack Query](https://tanstack.com/query) (server state)
 - [Zustand](https://zustand-demo.pmnd.rs/) (client state)
- UI Components: [shadcn/ui](https://ui.shadcn.com/) + [Radix UI](https://www.radix-ui.com/)
- Styling: [Tailwind CSS 4](https://tailwindcss.com/)
- Charts: [Recharts](https://recharts.org/)
- Forms: [React Hook Form](https://react-hook-form.com/) + [Zod](https://zod.dev/)

Backend

- Runtime: [Node.js 20](https://nodejs.org/) + [Express](https://expressjs.com/)
- Database: [SQLite](https://www.sqlite.org/)
- ORM: [Prisma 5](https://www.prisma.io/)
- Validation: [Zod](https://zod.dev/)
- File Upload: [Multer](https://github.com/expressjs/multer)
- Excel Parsing: [XLSX](https://sheetjs.com/)

Tooling

- Monorepo: [TurboRepo](https://turbo.build/repo)
- Package Manager: [pnpm 8](https://pnpm.io/)
- Linting: [ESLint 9](https://eslint.org/)
- Type Checking: [TypeScript 5.9](https://www.typescriptlang.org/)

Quick Start

Prerrequisitos

```
node --version # >= 18.0.0
pnpm --version # >= 8.0.0
```

Si no tienes pnpm instalado:

```
npm install -g pnpm@8
```

Instalación

```
# 1. Clonar el repositorio
git clone https://github.com/jeffrijol/finanzas-app.git
cd finanzas-app

# 2. Instalar dependencias
pnpm install

# 3. Configurar variables de entorno
# Backend (.env en packages/backend/)
echo "DATABASE_URL='file:../dev.db'" > packages/backend/.env
echo "PORT=3001" >> packages/backend/.env
echo "NODE_ENV=development" >> packages/backend/.env

# Frontend (.env.local en packages/frontend/)
echo "VITE_API_URL=http://localhost:3001/api" > packages/frontend/.env.local

# 4. Inicializar base de datos
cd packages/backend
pnpm prisma:generate
pnpm prisma:migrate
pnpm prisma:seed
cd ../../..

# 5. Iniciar servidor de desarrollo
pnpm dev
```

Acceso:

- 🖥️ **Frontend:** <http://localhost:4321> (<http://localhost:4321>)
- 🔑 **Backend API:** <http://localhost:3001/api> (<http://localhost:3001/api>)
- 📄 **Prisma Studio:** pnpm db:studio → <http://localhost:5555> (<http://localhost:5555>)

Estructura del Proyecto

```
finanzas-app/
├── packages/
│   ├── frontend/                # React + Vite application
│   │   ├── src/
│   │   │   ├── components/      # UI components (shadcn/ui)
│   │   │   ├── hooks/           # Custom React hooks
│   │   │   ├── lib/             # Utilities & API client
│   │   │   ├── pages/           # Route pages
│   │   │   ├── stores/          # Zustand stores
│   │   │   └── types/            # TypeScript definitions
│   │   ├── public/              # Static assets
│   │   └── package.json
│   └── backend/                 # Express API server
│       ├── src/
│       │   ├── controllers/      # Request handlers
│       │   ├── services/         # Business logic
│       │   ├── routes/           # API routes
│       │   ├── middleware/       # Express middleware
│       │   └── utils/            # Helper functions
│       ├── prisma/
│       │   ├── schema.prisma     # Database schema
│       │   ├── migrations/       # DB migrations
│       │   └── seed.ts           # Seed data
│       └── package.json
├── docs/                        # Documentation
│   ├── DEVELOPMENT.md           # Development guide
│   ├── ARCHITECTURE.md          # System architecture
│   ├── API.md                   # API reference
│   ├── FLUJO_USUARIO.md         # User flows
│   └── CI_CD_PLAN.md            # Future CI/CD setup
├── turbo.json                   # TurboRepo config
├── tsconfig.base.json           # Shared TypeScript config
├── package.json                 # Root package
└── README.md                    # This file
```

Scripts Disponibles

Root Level (ejecutar desde raíz del proyecto)

Script	Descripción
<code>pnpm dev</code>	Inicia frontend + backend en modo desarrollo
<code>pnpm build</code>	Construye ambos packages para producción
<code>pnpm lint</code>	Ejecuta linting en todo el monorepo
<code>pnpm clean</code>	Limpia builds y cache
<code>pnpm db:studio</code>	Abre Prisma Studio (gestión de BD)

Frontend (packages/frontend/)

Script	Descripción
<code>pnpm dev</code>	Servidor de desarrollo (Vite)
<code>pnpm build</code>	Build de producción
<code>pnpm preview</code>	Preview del build
<code>pnpm lint</code>	Lint del código frontend

Backend (packages/backend/)

Script	Descripción
<code>pnpm dev</code>	Servidor de desarrollo (tsx watch)
<code>pnpm build</code>	Compilar TypeScript
<code>pnpm start</code>	Ejecutar build compilado
<code>pnpm lint</code>	Lint del código backend
<code>pnpm prisma:generate</code>	Generar Prisma Client

`pnpm prisma:migrate` Ejecutar migraciones
`pnpm prisma:seed` Poblar BD con datos de prueba
`pnpm prisma:studio` Abrir Prisma Studio

Documentación

- [Guía de Desarrollo \(./docs/DEVELOPMENT.md\)](#): Setup detallado, troubleshooting, herramientas
 - [Arquitectura del Sistema \(./docs/ARCHITECTURE.md\)](#): Decisiones de diseño, estructura técnica
 - [Referencia de API \(./docs/API.md\)](#): Endpoints, schemas, ejemplos
 - [Flujos de Usuario \(./docs/FLUJO_USUARIO.md\)](#): Casos de uso y navegación
-

Roadmap

Implemented (v1.0)

- ☒ [x] Carga y parseo de archivos Excel/CSV
- ☒ [x] CRUD de transacciones con filtros avanzados
- ☒ [x] Gestión de Items y Categorías
- ☒ [x] Dashboard analítico con gráficos (Recharts)
- ☒ [x] Selector de período (trimestre/año)
- ☒ [x] Asignación manual y masiva de items
- ☒ [x] Generación de reportes PDF
- ☒ [x] Flujo borrador-confirmación para carga de archivos
- ☒ [x] Persistencia local de sesión de carga (LocalStorage)

In Progress (v1.1)

- ☐ [] Tests automatizados (Vitest)
- ☐ [] Integración de CI/CD (GitHub Actions)
- ☐ [] Optimización de performance (lazy loading, code splitting)

Planned (v2.0)

- ☐ [] Autenticación y multi-usuario
 - ☐ [] Exportación de reportes a Excel
 - ☐ [] Reglas de categorización automática (ML/regex)
 - ☐ [] Notificaciones de transacciones atípicas
 - ☐ [] Vista móvil optimizada
 - ☐ [] Temas (light/dark mode)
-

Contribuir

Para contribuir a este proyecto, consulta la guía detallada en el plan de CI/CD: [docs/CI_CD_PLAN.md \(./docs/CI_CD_PLAN.md\)](#)

Troubleshooting

Error: "Module not found"

```
rm -rf node_modules packages/*/node_modules
pnpm install
```

Error: Base de datos no encontrada

```
cd packages/backend
pnpm prisma:migrate
pnpm prisma:seed
```

Puerto 3001 o 4321 ocupado

```
# Windows
netstat -ano | findstr :3001
taskkill /PID <PID> /F

# Linux/Mac
lsof -ti:3001 | xargs kill -9
```

Para más problemas comunes, ver [docs/DEVELOPMENT.md](#) ([./docs/DEVELOPMENT.md#common-issues-solutions](#)).

Licencia

MIT © 2025

<p align="center">
Hecho con ❤️ usando React, Express y Prisma
</p>